Machine Learning Operations (MLOps) — AIMLCZG523

Assignment 01

Heart Disease Risk Prediction:

An End-to-End MLOps Pipeline from Data to a Monitored Kubernetes API

Student name	**Abhishek Kumar**
BITS ID	**2025AE05796**
Code repository	https://github.com/abhiparashar29/heart-disease-mlops
Pipeline video	https://drive.google.com/file/d/1Cch_7L05CpeeE6KBvFKDb3Ap_yB3pQiM/view?usp=sharing
API access	Local Kubernetes (minikube) — see Section 3 and README for access instructions

1. Introduction and objective

The goal of this assignment is to build a machine learning classifier that predicts the presence of heart disease from routine patient health data, and to deliver it the way a production team would: reproducible, tested, tracked, containerised, deployed on Kubernetes and monitored. The dataset is the **UCI Heart Disease (Cleveland)** dataset: 303 patients, 13 clinical features and a diagnosis column.

The final system downloads and cleans the data, tunes and compares three models with every run tracked in MLflow, packages the winning pipeline, verifies every code change with an automated GitHub Actions pipeline, serves predictions through a FastAPI service in Docker, runs that service on a local Kubernetes cluster behind a LoadBalancer and an Ingress, and monitors it with Prometheus and Grafana. The selected model, **Logistic Regression**, reached a cross-validated ROC-AUC of **0.905** and a test ROC-AUC of **0.960**, identifying 26 of 28 diseased patients in the test set. All evidence referenced in this report is also available as 28 screenshots in the screenshots/ folder of the repository.

Table 1: *Mapping of assignment tasks to implementation*

Assignment task	Where it is implemented	Evidence
1. Data acquisition & EDA	src/download_data.py, notebooks/01_eda.ipynb	Section 4, Figures 2–3
2. Feature engineering & models	src/preprocessing.py, src/train.py	Sections 5–6
3. Experiment tracking	MLflow (experiment heart-disease-classification)	Section 7, Figures 5–6
4. Packaging & reproducibility	models/, requirements*.txt, sklearn Pipeline	Section 8
5. CI/CD & testing	.github/workflows/ci.yml, tests/	Section 9, Figures 7–8
6. Containerisation	app/main.py, Dockerfile	Section 10, Figures 9–10
7. Production deployment	k8s/, scripts/deploy_local.sh	Section 11, Figures 11–12
8. Monitoring & logging	monitoring/, structured logs in app/main.py	Section 12, Figures 13–14

2. System architecture

The solution is organised in four layers (Figure 1). In the **development layer**, scripts download and clean the data, and train.py tunes three models and logs every run to MLflow; the best pipeline is exported as model.joblib with a metadata file. The **CI/CD layer** re-runs linting, the unit tests and training on GitHub's servers for every push, and fails the build if any step or the model-quality gate fails. The **serving layer** wraps the exported pipeline in a FastAPI service and packages it as a Docker image. The **deployment layer** runs two replicas of that image on

Kubernetes, exposes them through an Ingress and a LoadBalancer, and feeds their metrics into Prometheus and Grafana.

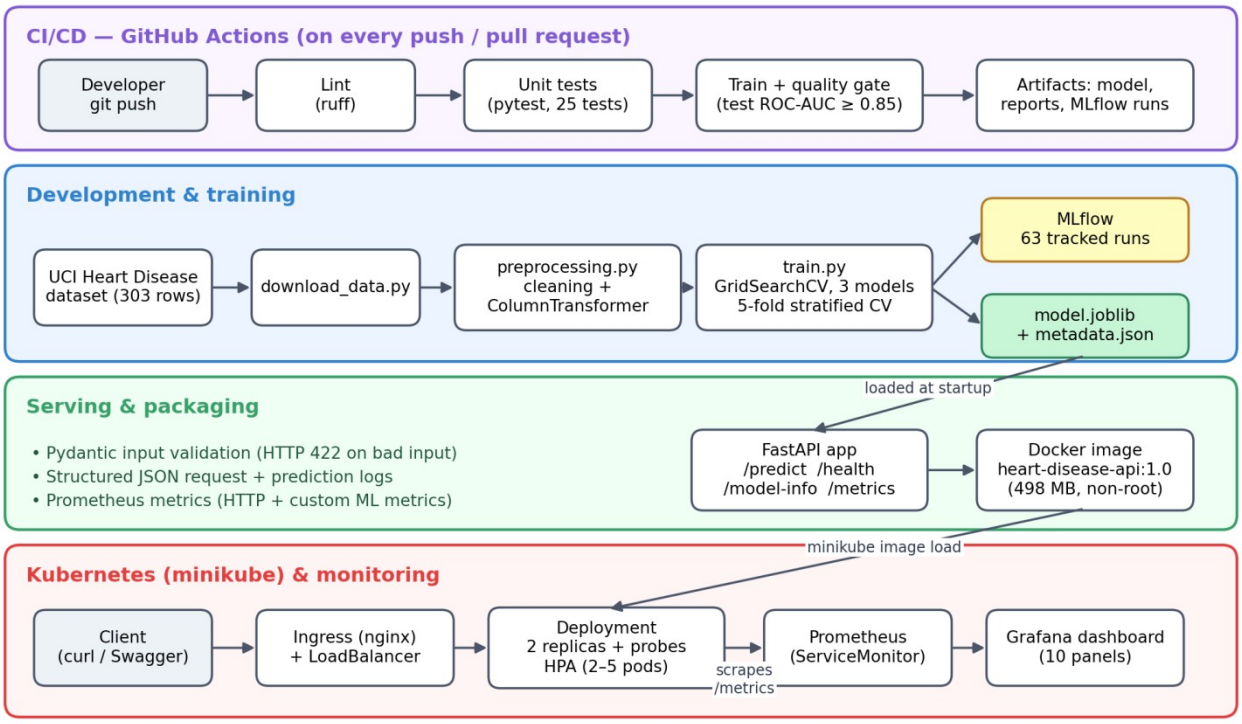


Figure 1: *End-to-end architecture of the heart disease MLOps pipeline*

3. Setup and installation

All work was carried out on the BITS Prayogshala virtual lab (Rocky Linux 9.5, 4 CPUs, 15 GB RAM). Because the lab VM is replaced between sessions and only ~/workspace persists, the conda environment, the Kubernetes CLI tools and a small start-up script were all stored in ~/workspace; everything else is recreated from the repository. The complete environment is pinned in requirements.txt (Python 3.11, scikit-learn 1.5.2, MLflow 2.17.2, FastAPI 0.115.4).

```
git clone https://github.com/abhiparashar29/heart-disease-mlops.git && cd heart-
disease-mlops
conda create -n mlops python=3.11 -y && conda activate mlops && pip install -r
requirements.txt
python src/download_data.py && python src/preprocessing.py && python -m src.train
pytest && ruff check src tests app            # 25 tests + linting
bash scripts/deploy_local.sh                   # minikube + API + Prometheus/Grafana
```

The deployment script starts minikube, builds the image, loads it into the cluster, applies the Kubernetes manifests, installs the monitoring stack and finishes with a smoke test against /health and /predict. The API is then reachable through the Ingress (curl --resolve "heart-api.local:80:$(minikube ip)" http://heart-api.local/predict) or through the LoadBalancer after running minikube tunnel. Full access instructions are in the repository README.

4. Data acquisition and exploratory data analysis

download_data.py fetches the dataset with the official ucimlrepo package (dataset id 45) and falls back to the raw UCI file if the package fails, saving data/raw/heart_raw.csv. The data has **303 rows, 14 columns and no duplicates**. Only **6 values are missing** (ca: 4, thal: 2), spread across both classes. The original target num ranges from 0 to 4; following the problem statement it was converted to a binary target (0 = no disease, 1 = disease), giving **164 (54.1%) vs 139 (45.9%)** — a mild imbalance that does not require resampling (Figure 2).

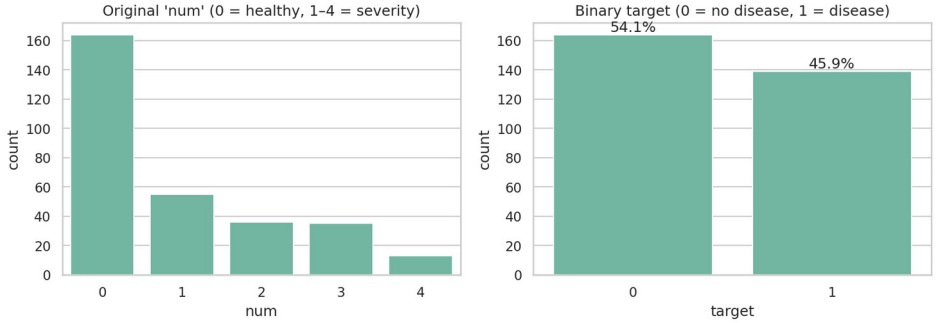
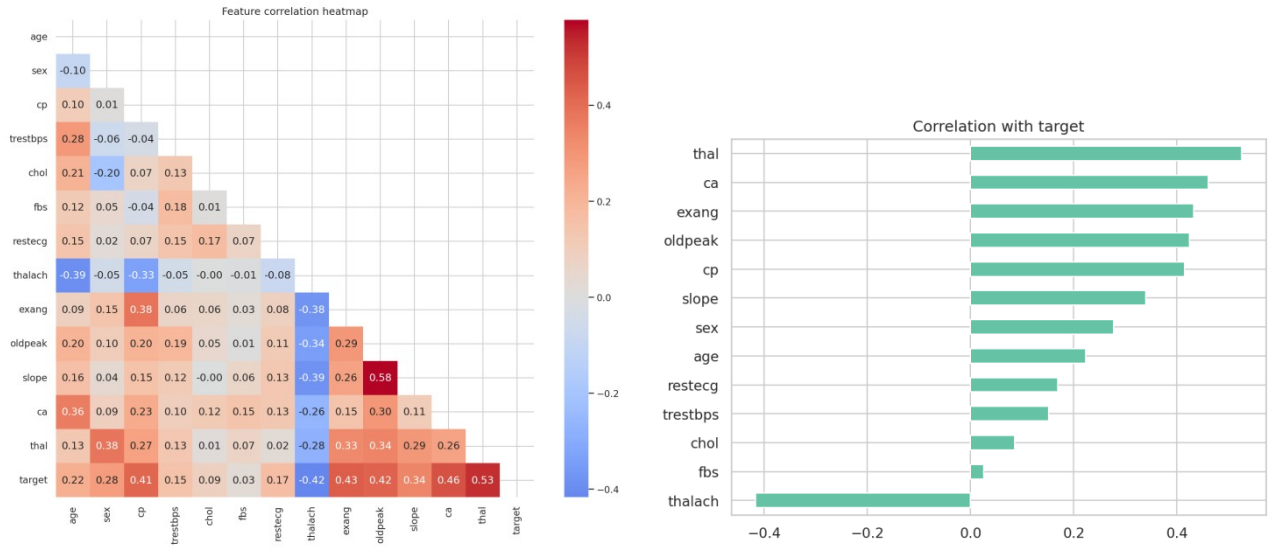


Figure 2: *Original severity scale (left) and binary target (right)*



(a) Feature correlation heatmap *(b) Correlation of each feature with the target*

Figure 3: *Correlation analysis*

The strongest relationships with the target are **thal (0.53), ca (0.46), exang (0.43), oldpeak (0.42), thalach (−0.42) and cp (0.41)**, while chol (0.09) and fbs (0.03) carry little signal (Figure 3). No pair of features exceeds 0.6 correlation (the highest is oldpeak–slope at 0.58), so no feature had to be dropped for multicollinearity. Because cp, restecg, slope and thal are nominal codes, their Pearson correlations are only indicative — a further reason to one-hot encode them. Outliers such as chol = 564 or oldpeak = 6.2 are clinically plausible and were kept.

Table 2: *How EDA findings shaped the modelling choices*

EDA finding	Modelling decision
6 missing values (2%) in categorical columns ca and thal	Impute with the mode **inside the pipeline** — no rows lost, no leakage, and the API can accept a missing value
Mild class imbalance (54 / 46)	No resampling; stratified splits and CV; report precision, recall and ROC-AUC, not only accuracy
Nominal codes (cp, restecg, slope, thal)	One-hot encoding with handle_unknown="ignore"
Very different scales (oldpeak 0–6 vs chol 126–564)	StandardScaler for numeric features
Small dataset (303 rows)	5-fold stratified CV; prefer simpler, regularised models
Medical context — missing a sick patient is costly	Track recall closely alongside ROC-AUC

5. Feature engineering and preprocessing

All preprocessing is defined once in src/preprocessing.py and shared by training, the unit tests and the API, so every stage transforms data identically. clean_data() validates the expected columns, removes duplicates, coerces invalid values to NaN and creates the binary target; it deliberately does **not** fill missing values. Imputation happens inside a scikit-learn ColumnTransformer that is placed in the same Pipeline as the model, so imputation

statistics are learned only from the training folds (no data leakage). The 13 raw features become **22 model features**.

Table 3: *Preprocessing pipeline*

Feature group	Columns	Transformation
Numeric	age, trestbps, chol, thalach, oldpeak	Median imputation → StandardScaler
Nominal	cp, restecg, slope, thal	Mode imputation → One-Hot Encoding (unknown categories ignored)
Binary	sex, fbs, exang	Mode imputation, kept as 0/1
Ordinal count	ca (0–3 vessels)	Mode imputation, kept numeric (order is meaningful)

6. Model development and evaluation

The data was split 80/20 (242 / 61 rows), stratified on the target with random_state = 42. Each candidate model was wrapped in the full preprocessing pipeline and tuned with GridSearchCV using **5-fold stratified cross-validation**, scoring accuracy, precision, recall, F1 and ROC-AUC, and refitting on **ROC-AUC**. The held-out test set was used only for the final evaluation.

Table 4: *Hyperparameter search space (60 combinations × 5 folds = 300 fits)*

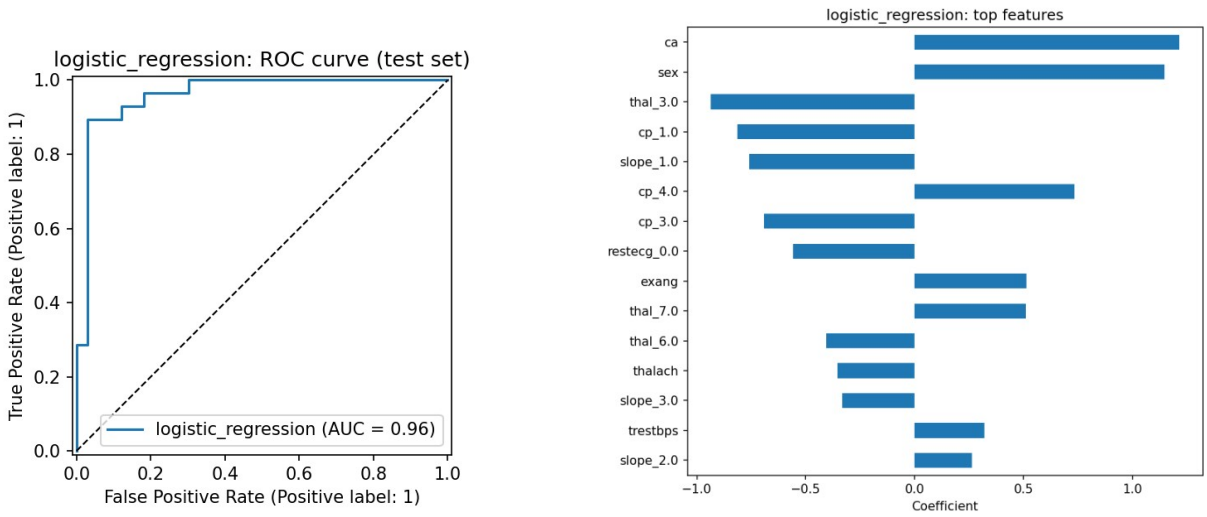
Model	Hyperparameter grid	Combinations
Logistic Regression	C ∈ {0.01, 0.1, 1, 10}; penalty ∈ {L1, L2}; class_weight ∈ {None, balanced}	16
Random Forest	n_estimators ∈ {100, 300}; max_depth ∈ {None, 4, 8}; min_samples_leaf ∈ {1, 3, 5}; max_features ∈ {sqrt, 0.5}	36
Gradient Boosting	n_estimators ∈ {100, 200}; learning_rate ∈ {0.05, 0.1}; max_depth ∈ {2, 3}	8

Table 5: *Model comparison (best configuration of each model)*

Model	CV ROC-AUC	Test ROC-AUC	Test accuracy	Test precision	Test recall
Logistic Regression	**0.905 ± 0.019**	**0.960**	0.869	0.813	0.929
Random Forest	0.901 ± 0.024	0.958	0.869	0.813	0.929
Gradient Boosting	0.874 ± 0.028	0.946	0.885	0.839	0.929

Logistic Regression (C = 1.0, L2, no class weighting) was selected. It achieved the highest CV ROC-AUC with the lowest variance across folds. Random Forest is statistically tied (a 0.004 difference, well inside one standard deviation), so the simpler model was preferred: it is interpretable — its coefficients explain which factors raise or lower risk, which matters in a medical setting — and it is small and fast to serve. The model was chosen on the **CV score rather than the test score** so that the test set remains an unbiased check; test scores are slightly higher than CV scores only because the 61-row test set is small and therefore noisy.

On the test set the model identifies **26 of 28 diseased patients (recall 0.93)** with 6 false alarms among 33 healthy patients (Figure 4). The coefficients agree with the EDA: more blocked vessels (ca), male sex, asymptomatic chest pain (cp = 4), exercise angina and a reversible thalassemia defect (thal = 7) raise the predicted risk, while a normal thal result and a high maximum heart rate lower it. The tuning results also showed why tuning matters: with C = 0.01 and an L1 penalty, regularisation removed every coefficient and the model scored a CV ROC-AUC of exactly 0.5 — no better than chance.



(a) ROC curve on the test set (AUC = 0.96) *(b) Largest model coefficients*

Figure 4: *Evaluation of the selected Logistic Regression model*

7. Experiment tracking with MLflow

Every training run is recorded in the MLflow experiment heart-disease-classification. Each model has a parent run, and every hyperparameter combination is logged as a **nested child run** — 3 parent and 60 child runs, **63 in total** (Figure 5). Parent runs record the best parameters, the CV metrics as mean and standard deviation, the test metrics, the confusion matrix, ROC curve and feature-importance plots, the classification report, the full grid-search table and the model itself with an input signature and example. The selected run is tagged selected_as_best = true, which makes the automatic choice auditable (Figure 6).

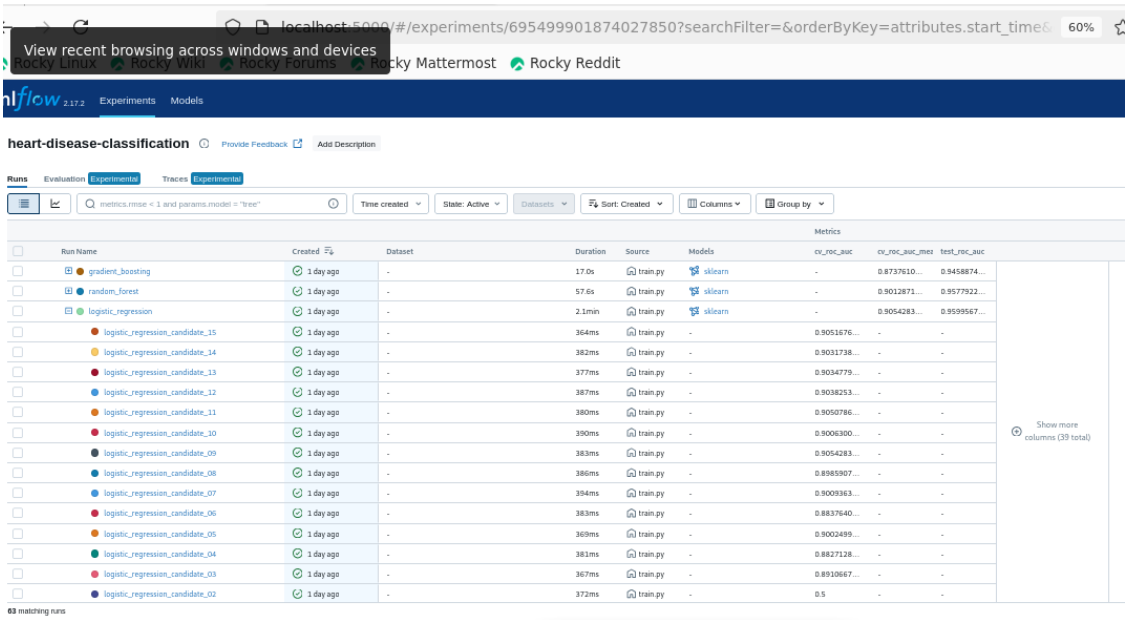
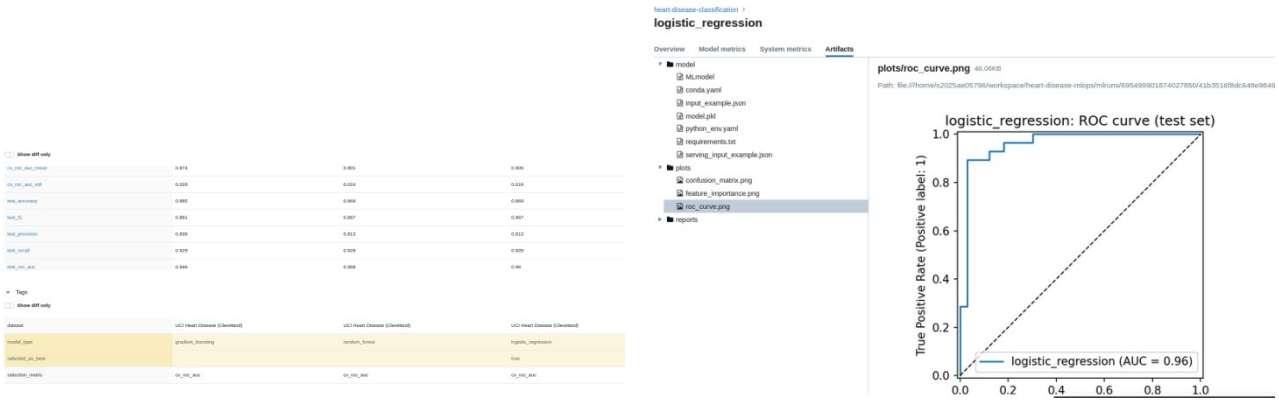


Figure 5: *MLflow run list: three parent runs and nested candidate runs with their CV ROC-AUC (63 runs)*



(a) Side-by-side metrics and selected_as_best tag *(b) Logged artifacts: packaged model and plots*

Figure 6: *MLflow run comparison and artifacts*

8. Model packaging and reproducibility

- **Single reusable artifact:** models/model.joblib contains the complete pipeline (imputation, scaling, encoding and model), so raw patient data goes in and a prediction comes out — no preprocessing code has to be re-implemented for serving.
- **Metadata:** models/metadata.json records the model name, MLflow run ID, best parameters, CV and test metrics, feature list, training timestamp and the scikit-learn and Python versions.
- **MLflow model format:** the same pipeline is also logged in MLflow with conda.yaml, python_env.yaml, requirements.txt and an input example (Figure 6b).
- **Pinned environments:** requirements.txt pins every library for training; requirements-serve.txt contains only the eight packages needed for serving, with identical versions so the model always loads with the scikit-learn version that trained it.
- **Determinism:** random_state = 42 for the split, the CV folds and the models; a unit test checks that predictions are identical across calls.

9. Automated testing and CI/CD

The test suite contains **25 pytest tests**, and ruff enforces linting and import order on src, tests and app.

Table 6: *Unit test coverage*

Test file	Tests	What is verified
test_preprocessing.py	10	Dataset shape; binary target mapping (0→0, 1–4→1); duplicate removal; '?' converted to NaN; error on missing columns; input not mutated; 22 output features with no NaNs; numeric scaling; unseen categories handled
test_model.py	6	A fresh pipeline beats chance (AUC > 0.8); saved model returns valid probabilities, handles missing values, is deterministic; **a high-risk patient (from the EDA patterns) scores higher than a low-risk one**; metadata matches the feature list
test_api.py	9	Root and health endpoints; prediction and confidence returned; missing optional fields accepted; out-of-range, missing and invalid values rejected with 422; custom metrics exposed; request-ID header present

The GitHub Actions workflow (.github/workflows/ci.yml) runs on every push and pull request to main as three chained jobs: **Lint → Unit tests → Train & evaluate**. Each job runs only if the previous one succeeded. The training job re-runs preprocessing and training from scratch, applies a **quality gate** that fails the pipeline if the test ROC-AUC drops below 0.85, writes a model-comparison table to the job summary, and uploads the trained model, training reports, MLflow runs and JUnit test results as artifacts (kept 14 days). A concurrency rule cancels outdated runs when newer commits arrive. Figure 7 shows a complete successful run.

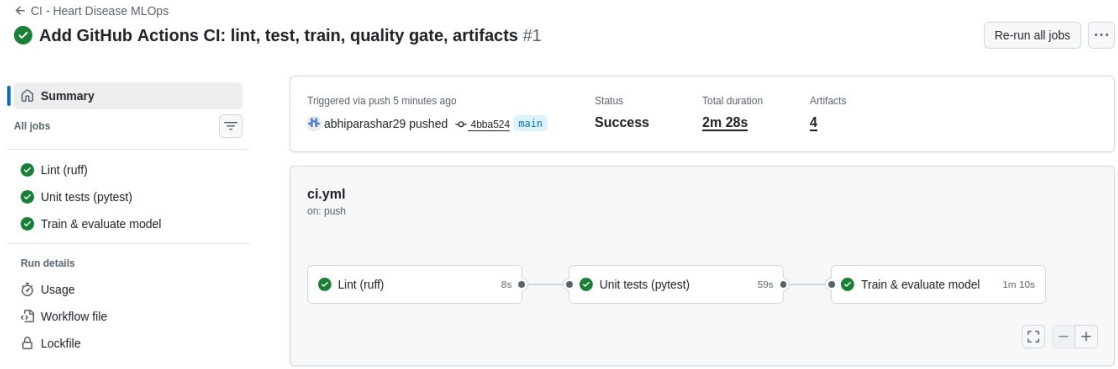
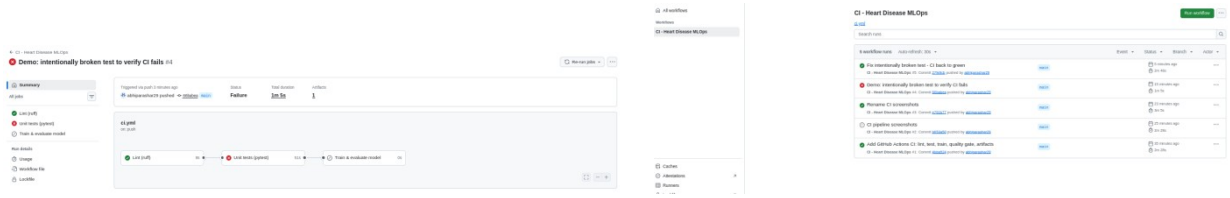


Figure 7: *Successful pipeline run: lint, tests and training in 2 min 28 s with four artifacts*

To prove that the pipeline blocks faulty changes, a test was deliberately broken and pushed. The test job failed with a clear AssertionError, the training job was **skipped**, and pushing the fix turned the pipeline green again (Figure 8).



(a) Deliberately broken test: training skipped *(b) Run history: pass → fail → fixed*

Figure 8: *Failure demonstration — the pipeline fails on test errors*

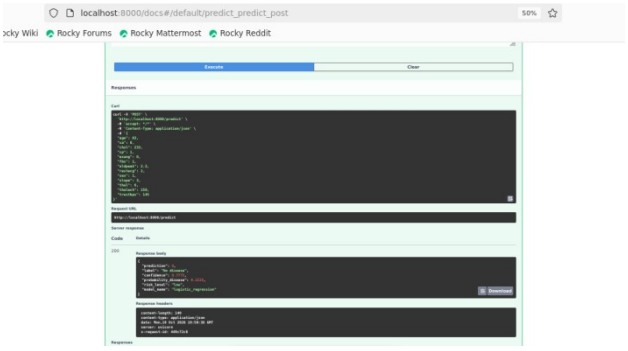
10. Model serving API and containerisation

app/main.py implements the service with FastAPI. The model is loaded once at start-up. Every input field is validated by a Pydantic schema with medical-range limits (for example age 1–120 and thal ∈ {3, 6, 7}); ca and thal are optional because they can be missing in real data and the pipeline imputes them. Invalid requests receive **HTTP 422** with a clear error instead of a wrong prediction. The response contains the prediction, a label, the **confidence** (probability of the predicted class), the probability of disease and a risk level (low < 0.3 ≤ moderate < 0.7 ≤ high).

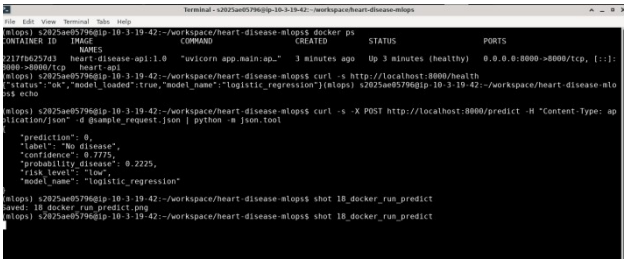
Table 7: *API endpoints*

Endpoint	Purpose
GET /health	Liveness/readiness check (model loaded?) — used by Docker and Kubernetes
GET /model-info	Metadata and metrics of the loaded model
POST /predict	JSON patient record in → prediction, confidence, probability, risk level out
GET /metrics	Prometheus metrics: HTTP metrics plus custom prediction metrics

The Dockerfile uses the python:3.11-slim base image and installs requirements-serve.txt in a separate layer, so code changes rebuild in seconds. The image copies only src, app and models, runs as a **non-root user (UID 1000)** and defines a HEALTHCHECK against /health. The final image is **498 MB**. The container built and served predictions identical to the local run (Figure 9); its logs show the model loading from /app/models, Docker's health checks every 30 seconds, and structured JSON logs for each request and prediction with a request ID and latency (Figure 10).



(a) Swagger UI: /predict request and response



(b) Container healthy; /health and /predict via curl

Figure 9: *API serving locally and inside Docker*

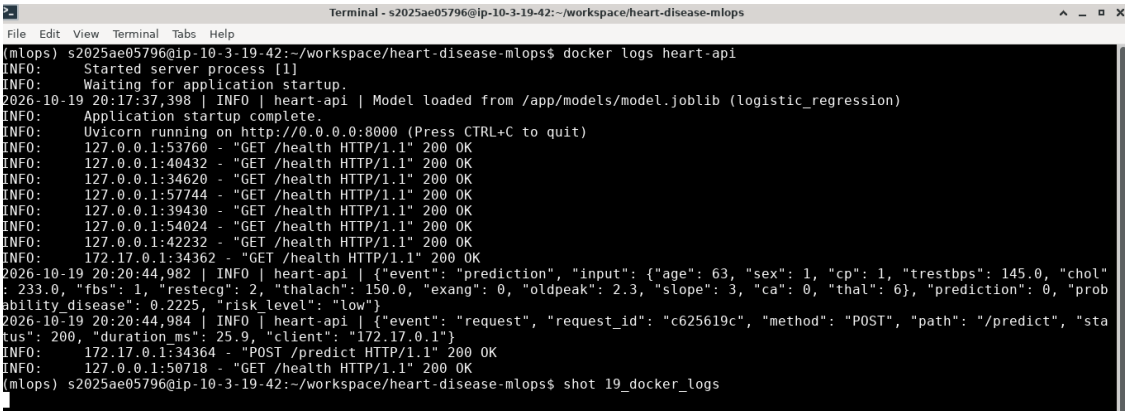


Figure 10: *Container logs: model loading, health checks and structured request/prediction logs*

11. Production deployment on Kubernetes

The image was deployed on a local Kubernetes cluster (minikube v1.35, Kubernetes v1.32) using the manifests in k8s/:

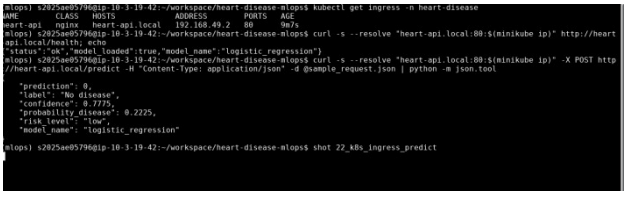
Table 8: *Kubernetes manifests*

Manifest	Configuration
00-namespace.yaml	Dedicated namespace heart-disease
01-deployment.yaml	2 replicas; rolling updates with maxUnavailable 0 (zero downtime); readiness and liveness probes on /health; CPU/memory requests and limits; non-root security context; Prometheus annotations
02-service.yaml	Service of type **LoadBalancer** (port 80 → 8000), exposed with minikube tunnel
03-ingress.yaml	NGINX **Ingress** for host heart-api.local
04-hpa.yaml	Horizontal Pod Autoscaler: 2–5 replicas at 70% CPU (bonus)

The first rollout failed with CreateContainerConfigError: the deployment requires runAsNonRoot, but the image declared its user by name (appuser), which Kubernetes cannot verify. Declaring the numeric runAsUser: 1000 fixed it, and the rolling update replaced the failed ReplicaSet without downtime (Figure 11a). The endpoints were verified through both the Ingress (Figure 11b) and the LoadBalancer (Figure 12), returning the same prediction. Sending 20 requests showed the Service load-balancing them exactly **11 / 11** across the two pods.



(a) Pods, LoadBalancer service, deployment, ReplicaSets and HPA



(b) Prediction through the Ingress

Figure 11: *Kubernetes deployment and endpoint verification*

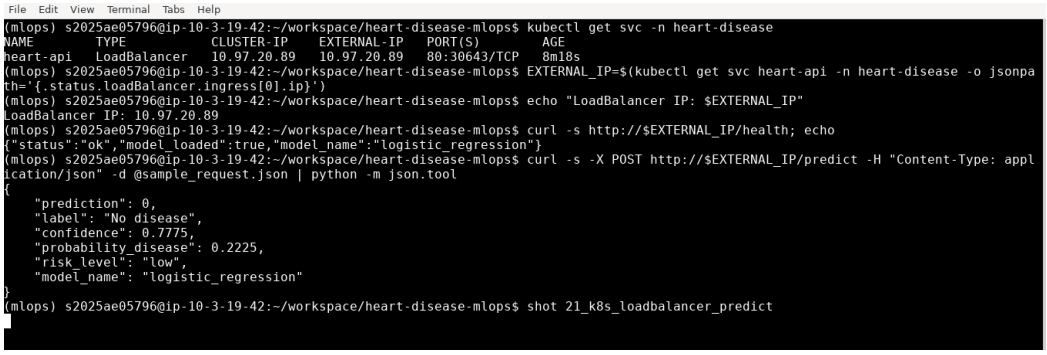


Figure 12: *Prediction through the LoadBalancer external IP (minikube tunnel)*

12. Monitoring and logging

Logging. A middleware logs every request as structured JSON (request ID, method, path, status, latency, client); each prediction is logged with its input, output and probability. Kubernetes health checks and Prometheus scrapes are logged at DEBUG level so they do not drown out real traffic. The X-Request-ID response header links a client response to its log line.

Metrics. The API exposes default HTTP metrics (requests by endpoint and status, latency histograms) plus two custom ML metrics: heart_predictions_total, counted per predicted class, and heart_disease_probability, a histogram of predicted probabilities. Prometheus and Grafana were installed with the kube-prometheus-stack Helm chart; a ServiceMonitor makes Prometheus scrape both pods every 15 seconds (Figure 13). The Grafana dashboard is **provisioned as code** from a ConfigMap, so it is version-controlled and recreated automatically.

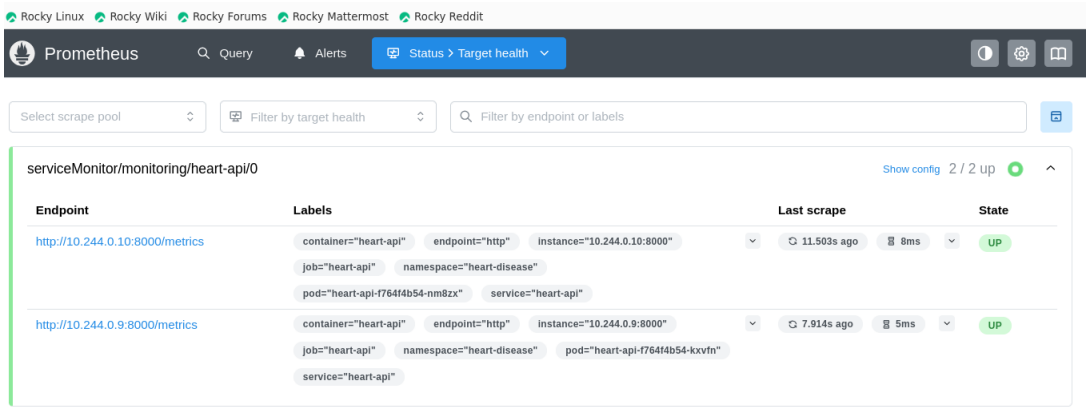


Figure 13: *Prometheus target health: both API pods discovered and UP*

A traffic generator (monitoring/generate_traffic.sh) sent 300 requests with random patient profiles, every 10th deliberately invalid. The dashboard (Figure 14) showed 2 pods up, 292 predictions, 62.7% predicted as disease, the rejected requests, two equal per-pod request lines confirming load balancing, stable memory of about 178 MB per pod, and the distribution of predicted probabilities — the panel that would reveal **prediction drift** if incoming data changed. Request logs show actual /predict latencies of about 20–25 ms; the dashboard's p50/p95 values (≈ 50 / 95 ms) are interpolated from coarse default histogram buckets and therefore overstate latency.



Figure 14: *Grafana dashboard "Heart Disease API – Monitoring" after 300 generated requests*

13. Challenges and solutions

Table 9: *Challenges faced and how they were solved*

Challenge	Solution
Lab VM disconnects and is replaced between sessions; only ~/workspace persists	Conda env (-p ~/workspace/envs), CLI tools and repo-local git settings stored in ~/workspace; start.sh restores a session in one command; tmux for long-running jobs; frequent pushes to GitHub
Kubernetes refused to start pods (runAsNonRoot with a named user)	Declared numeric runAsUser: 1000 in the deployment security context
Container memory panel empty (cAdvisor labels missing with minikube's Docker driver)	Used the API's own process_resident_memory_bytes metric instead
Large serving image with the full training environment	Separate requirements-serve.txt and .dockerignore → 498 MB image
Overlapping CI runs when pushing quickly	Concurrency rule that cancels outdated runs

14. Conclusion and future work

The project delivers a complete, reproducible MLOps workflow: a validated preprocessing pipeline, three tuned and fully tracked models, an interpretable Logistic Regression model with ROC-AUC 0.96 and recall 0.93, 25 automated tests in a CI pipeline with a quality gate, a validated and observable FastAPI service in a 498 MB Docker image, and a Kubernetes deployment with health probes, autoscaling, load balancing and a Prometheus/Grafana dashboard.

Natural next steps are: pushing the image to a container registry and deploying from CI (continuous deployment); registering models in the MLflow Model Registry; adding Grafana alerts for error rate and probability drift; finer latency histogram buckets; and validating the model on the other UCI centres (Hungary, Switzerland, Long Beach) to test how well it generalises beyond Cleveland.